

Uni-App 通用卡片组件：Props控制差异化展示

在Uni-App开发中，“一次封装、多场景复用”是组件化的核心目标。通过为通用组件设计灵活的Props参数，可实现“传递不同属性值，展示不同效果”的需求，避免重复开发相似组件。本文以高频使用的“通用卡片组件”为例，从Props设计、组件实现、多场景调用三个层面，详细讲解如何通过Props精准控制组件的视觉风格、布局结构与功能表现。

一、核心设计思路：Props参数体系构建

通用卡片需适配“商品展示、资讯推荐、个人信息、功能入口”等多场景，核心是围绕“差异化维度”设计Props参数。遵循“基础属性统一、扩展属性可选”的原则，将Props分为四类，确保参数体系清晰且灵活：

1. Props参数分类与说明

参数分类	核心参数	类型	默认值
基础样式	cardStyle	String	"default"
	borderRadius	String	"20rpx"
	shadow	Boolean	true
	bgColor	String	""
布局结构	layout	String	"vertical"
	imgPosition	String	"top"
	padding	String	"20rpx"
内容配置	title	String	""

	desc	String	""
	imgUrl	String	""
功能控制	showArrow	Boolean	false
	showTag	Boolean	false
	tagText	String	""

2. 设计核心原则

- **默认值合理**：核心参数设置符合高频场景的默认效果（如默认垂直布局、显示阴影），减少常规使用时的参数传递；
- **优先级清晰**：自定义属性（如bgColor）优先级高于风格属性（cardStyle），支持精细化样式调整；
- **场景化适配**：通过“布局+风格+功能”的参数组合，覆盖主流卡片使用场景，避免参数冗余；
- **类型校验**：为Props添加类型限制（如Boolean、String），避免传入错误参数导致组件异常。

二、通用卡片组件实现（Props驱动）

基于上述Props设计，封装名为 `CommonCard` 的通用组件，通过条件渲染与动态样式绑定，实现不同属性值对应的差异化展示效果，同时融入图片容错、文字省略等细节处理。

1. 组件完整代码（components/CommonCard/CommonCard.vue）

Code block

```
1  <template>
2    <!-- 卡片容器：通过Props动态绑定样式 -->
3    <navigator
4      :url="link"
5      open-type="navigate"
6      class="common-card"
7      :class="{ 'card-no-shadow': !shadow}"
8      :style="[
9        baseStyle,
10       customBgStyle,
11       {padding: padding, borderRadius: borderRadius}
12     ]"
13    >
```

```
14 <!-- 左上角标签: showTag为true时显示 -->
15 <view
16   class="card-tag"
17   v-if="showTag && tagText"
18   :style="tagStyle"
19 >
20   {{tagText}}
21 </view>
22
23 <!-- 垂直布局: 图片在上, 文字在下 -->
24 <view class="card-content vertical-layout" v-if="layout === 'vertical'">
25   <!-- 图片区域: imgUrl不为空时显示 -->
26   <view class="card-img-container" v-if="imgUrl">
27     <image
28       :src="imgUrl"
29       class="card-img"
30       mode="cover"
31       lazy-load="true"
32       @error="handleImgError"
33     ></image>
34   </view>
35
36   <!-- 文字区域 -->
37   <view class="card-text" :style="{marginTop: imgUrl ? '16rpx' : 0}">
38     <text class="card-title" :style="titleStyle">{{title}}</text>
39     <text
40       class="card-desc"
41       :style="descStyle"
42       v-if="desc"
43       :title="desc"
44     >{{desc}}</text>
45   </view>
46
47   <!-- 右侧箭头: showArrow为true时显示 -->
48   <uni-icons
49     class="card-arrow"
50     type="right"
51     size="24rpx"
52     color="#999"
53     v-if="showArrow"
54   ></uni-icons>
55 </view>
56
57 <!-- 水平布局: 图片在左/右, 文字在另一侧 -->
58 <view class="card-content horizontal-layout" v-if="layout ===
'horizontal'">
```

```

59     <view class="card-img-container horizontal-img" v-if="imgUrl" :style="
{order: imgPosition === 'right' ? 2 : 1}">
60         <image
61             :src="imgUrl"
62             class="card-img horizontal-img-item"
63             mode="cover"
64             lazy-load="true"
65             @error="handleImgError"
66         ></image>
67     </view>
68
69     <view
70         class="card-text horizontal-text"
71         :style="[
72             {flex: 1},
73             {marginLeft: imgPosition === 'left' && imgUrl ? '16rpx' : 0},
74             {marginRight: imgPosition === 'right' && imgUrl ? '16rpx' : 0}
75         ]"
76     >
77         <text class="card-title" :style="titleStyle">{{title}}</text>
78         <text
79             class="card-desc"
80             :style="descStyle"
81             v-if="desc"
82             :title="desc"
83         >{{desc}}</text>
84     </view>
85

```

2. 组件核心逻辑解析

(1) Props驱动的条件渲染

通过 `v-if="layout === 'vertical'"` 等条件判断，根据Props中 `layout` 属性值渲染不同布局结构，实现“垂直/水平/网格”三种布局的切换。同时，通过 `v-if="imgUrl"` 控制图片显示，`v-if="showTag"` 控制标签显示，实现内容的按需渲染。

(2) 动态样式绑定技巧

利用Vue的动态样式绑定（`:style`），结合计算属性将Props转换为具体样式：

- **基础风格转换：** `baseStyle` 计算属性根据 `cardStyle` 值生成对应的背景色、边框样式；
- **优先级处理：** `customBgStyle` 计算属性判断是否有自定义背景色，优先级高于基础风格；
- **场景化适配：** 标题、描述、标签的颜色会根据 `cardStyle` 自动适配（如主题色卡的文字为白色），同时支持通过Props自定义颜色覆盖默认值。

(3) 参数校验保障稳定性

为 `cardStyle`、`layout` 等核心Props添加 `validator` 校验函数，限制仅能传入预设值（如 `['default', 'primary']`），避免因传入错误参数导致组件布局混乱，提升组件稳定性。

三、多场景调用示例：Props差异化传递

在页面中引入CommonCard组件，通过传递不同的Props属性值，实现商品卡片、资讯卡片、功能入口卡片等不同效果，充分体现组件的复用价值。

1. 场景一：商品卡片（垂直布局+主题色+标签）

需求：电商场景商品展示，垂直布局，突出“新品”标签，主题色风格，点击跳转商品详情。

Code block

```
1  <template>
2    <view class="product-page">
3      <CommonCard
4        cardStyle="primary"
5        layout="vertical"
6        imgUrl="https://example.com/product1.jpg"
7        title="2024秋季新款休闲外套"
8        desc="宽松版型，舒适面料，多色可选，适合日常穿搭"
9        link="/pages/productDetail/productDetail?id=1"
10       showTag
11       tagText="新品"
12       showArrow
13       borderRadius="16rpx"
14     />
15   </view>
16 </template>
17
18 <script>
19   import CommonCard from '@components/CommonCard/CommonCard.vue';
20   export default { components: { CommonCard } }
21 </script>
```

2. 场景二：资讯卡片（水平布局+左侧图片+箭头）

需求：资讯列表场景，水平布局，图片在左侧，显示右侧箭头提示跳转，简约边框风格。

Code block

```
1  <template>
2    <view class="news-page">
3      <CommonCard
4        cardStyle="outline"
5        layout="horizontal"
```

```

6      imgPosition="left"
7      imgUrl="https://example.com/news1.jpg"
8      title="Uni-App 3.0版本发布，支持更多跨端能力"
9      desc="新增XX功能，性能提升30%，开发者可快速升级体验"
10     link="/pages/newsDetail/newsDetail?id=101"
11     showArrow
12     shadow="false"
13     padding="16rpx"
14   />
15 </view>
16 </template>
17
18 <script>
19   import CommonCard from '@components/CommonCard/CommonCard.vue';
20   export default { components: { CommonCard } }
21 </script>

```

3. 场景三：功能入口卡片（网格布局+磨砂风格）

需求：个人中心功能入口，网格布局，磨砂玻璃风格，无阴影，点击跳转对应功能页。

Code block

```

1   <template>
2     <view class="user-page">
3       <CommonCard
4         cardStyle="frosted"
5         layout="grid"
6         imgUrl="/static/icons/collect.png"
7         title="我的收藏"
8         link="/pages/collect/collect"
9         shadow="false"
10        borderRadius="50%"
11        padding="0"
12        titleColor="#333333"
13      />
14    </view>
15  </template>
16
17  <script>
18    import CommonCard from '@components/CommonCard/CommonCard.vue';
19    export default { components: { CommonCard } }
20  </script>

```

4. 场景四：自定义样式卡片（覆盖默认属性）

需求：特殊活动场景，自定义背景色与文字色，不遵循预设风格，实现个性化展示。

Code block

```
1  <template>
2    <view class="activity-page">
3      <CommonCard
4        bgColor="#F0F9FF"
5        layout="vertical"
6        imgUrl="https://example.com/activity.jpg"
7        title="双11预售开启，付定金立减50元"
8        desc="活动时间：10月20日-10月31日，限前1000名"
9        link="/pages/activity/activity"
10       showTag
11       tagText="限时"
12       tagBgColor="#4096ff"
13       tagColor="#FFFFFF"
14       titleColor="#1890ff"
15       descColor="#69b1ff"
16       showArrow
17     />
18   </view>
19 </template>
20
21 <script>
22   import CommonCard from '@components/CommonCard/CommonCard.vue';
23   export default { components: { CommonCard } }
24 </script>
```

四、Props设计与使用的最佳实践

1. Props设计技巧

- **分类命名**：按“样式-布局-内容-功能”分类命名Props（如 `cardStyle` 属于样式类，`layout` 属于布局类），提升可读性；
- **预设枚举值**：核心风格、布局等参数采用预设枚举值（如`cardStyle`的四个选项），避免自由传值导致的样式混乱；
- **冗余控制**：避免设计过于细化的Props（如单独控制标题字体粗细），可通过“自定义样式参数+默认适配”平衡灵活性与简洁性。

2. 组件使用注意事项

- **优先用预设值**：常规场景优先使用组件预设的Props值（如`cardStyle="default"`），减少自定义参数传递；

- **明确跳转链接：**需要点击跳转时，务必传递 `link` 属性，并确保链接在pages.json中注册（小程序端）；
- **图片地址规范：**网络图片需使用HTTPS协议（小程序端要求），本地图片建议放在static目录下。



扩展思路：若需实现更复杂的差异化效果（如卡片加载状态、点击反馈差异），可在现有Props基础上增加 `loading`（加载状态）、`clickType`（点击类型）等参数，通过条件渲染与事件分发实现功能扩展，进一步提升组件的通用性。

（注：文档部分内容可能由 AI 生成）